

Abstract State Machines

Compilers

Consider the following extension of the λ -calculus with let-expressions:

$$M \in \mathbf{LC} ::= x \mid \lambda x.M \mid M \ M \mid \mathbf{let} \ x = M \ \mathbf{in} \ N$$

These let-expressions are very simple: $\mathbf{let} \ x = P \ \mathbf{in} \ N$ behaves as $(\lambda x.N) \ P$, i.e., P is lazily evaluated.

1. Provide a call-by-name interpreter for this language.
2. Apply the closure-conversion transformation to your evaluator.
3. Apply the CPS transformation to your evaluator.
4. Apply defunctionalization to your evaluator.
5. Turn your evaluator into a “high-level” Krivine machine (1st lecture).
6. Turn your “high-level” Krivine machine into a “low-level” Krivine machine (2nd lecture).
7. Optimize your machine to avoid short-lived closures on the stack.

Solution

1. Here is an interpreter for this language:

$$\begin{aligned}
 \text{eval}(x) &= x \\
 \text{eval}(\lambda x.M) &= \lambda x.M \\
 \text{eval}(M \ N) &= \text{eval}(P[x \setminus N]) \\
 &\quad \text{where } \text{eval}(M) = \lambda x.P \\
 \text{eval}(\text{let } x = P \text{ in } N) &= \text{eval}(N[x \setminus P])
 \end{aligned}$$

2. Here is the closure conversion:

$$\begin{aligned}
 \text{cc}(x, e) &= \text{cc}(M, e') \\
 &\quad \text{where } e(x) = \langle M, e' \rangle \\
 \text{cc}(\lambda x.M, e) &= \langle \lambda x.M, e \rangle \\
 \text{cc}(M \ N, e) &= \text{cc}(P, e'[x \mapsto \langle N, e \rangle]) \\
 &\quad \text{where } \text{cc}(M, e) = \langle \lambda x.P, e' \rangle \\
 \text{cc}(\text{let } x = P \text{ in } N, e) &= \text{cc}(N, e[x \mapsto \langle P, e \rangle])
 \end{aligned}$$

3. Here is the CPS transformation:

$$\begin{aligned}
 \text{cont}(x, e, k) &= \text{cont}(M, e', k) \\
 &\quad \text{where } e(x) = \langle M, e' \rangle \\
 \text{cont}(\lambda x.M, e, k) &= k \ \langle \lambda x.M, e \rangle \\
 \text{cont}(M \ N, e, k) &= \text{cont}(M, e, k') \\
 &\quad \text{where } k' = \text{fun } \langle \lambda x.P, e' \rangle \Rightarrow \text{cont}(P, e'[x \mapsto \langle N, e \rangle], k) \\
 \text{cont}(\text{let } x = P \text{ in } N, e, k) &= \text{cont}(N, e'[x \mapsto \langle P, e \rangle], k)
 \end{aligned}$$

4. Here is the evaluator after applying defunctionalization:

$$\begin{aligned}
 \text{defunc}(x, e, k) &= \text{defunc}(M, e', k) \\
 &\quad \text{where } e(x) = \langle M, e' \rangle \\
 \text{defunc}(\lambda x.M, e, k) &= \text{apply_cont}(\lambda x.M, e, k) \\
 \text{defunc}(M \ N, e, k) &= \text{defunc}(M, e, \text{CONT_}\lambda(N, e, k)) \\
 \text{defunc}(\text{let } x = P \text{ in } N, e, k) &= \text{defunc}(N, e[x \mapsto \langle P, e \rangle], k) \\
 \text{apply_cont}(M, e, \text{CONT_}I) &= \langle M, e \rangle \\
 \text{apply_cont}(\lambda x.P, e', \text{CONT_}\lambda(N, e, k)) &= \text{defunc}(P, e'[x \mapsto \langle N, e \rangle], k)
 \end{aligned}$$

5. Here is the Krivine machine:

$$\begin{aligned}
 \text{asm}(x, e, k) &= \text{asm}(M, e', k) \\
 &\quad \text{where } e(x) = \langle M, e' \rangle \\
 \text{asm}(\lambda x.M, e, []) &= \langle \lambda x.M, e \rangle \\
 \text{asm}(\lambda x.M, e, \langle N, e' \rangle :: k) &= \text{asm}(M, e[x \mapsto \langle N, e' \rangle], k) \\
 \text{asm}(M \ N, e, k) &= \text{asm}(M, e, \langle N, e \rangle :: k) \\
 \text{asm}(\text{let } x = P \text{ in } N, e, k) &= \text{asm}(N, e[x \mapsto \langle P, e \rangle], k)
 \end{aligned}$$

6. Finally, to use the low-level 3-instruction Krivine abstract machine, we can translate this language as follows:

$$\begin{aligned}
 \llbracket n \rrbracket &= \text{access}(n) \\
 \llbracket \lambda M \rrbracket &= \text{grab}; \llbracket M \rrbracket \\
 \llbracket M \ N \rrbracket &= \text{push}(N); \llbracket M \rrbracket \\
 \llbracket \text{let } P \text{ in } N \rrbracket &= \text{push}(P); \text{grab}; \llbracket N \rrbracket
 \end{aligned}$$

where `let P in N` is the de Bruijn version of let-expressions.

7. To avoid short-lived closures on the stack we extend the language as follows:

- `push(c)`: to push an instruction onto the stack
- `grab`: to move a closure from the stack to the heap
- `access(n)`: to access a closure from the heap
- `store(c)`: to push a closure directly onto the heap
- plus sequencing: $c_1; c_2$

We translate our language as follows:

$$\begin{aligned}
 \llbracket n \rrbracket &= \text{access}(n) \\
 \llbracket \lambda M \rrbracket &= \text{grab}; \llbracket M \rrbracket \\
 \llbracket M \ N \rrbracket &= \text{push}(\llbracket N \rrbracket); \llbracket M \rrbracket \\
 \llbracket \text{let } P \text{ in } N \rrbracket &= \text{store}(\llbracket P \rrbracket); \llbracket N \rrbracket
 \end{aligned}$$

The machine is now:

$$\begin{aligned}
 K(\text{access}(0); c, \langle c', e' \rangle :: e, k) &= K(c', e', k) \\
 K(\text{access}(n+1); c, \langle c', e' \rangle :: e, k) &= K(n, e, k) \\
 K(\text{grab}; c, e, []) &= \langle \text{grab}; c, e \rangle \\
 K(\text{grab}; c, e, \langle c', e' \rangle :: s) &= K(c, \langle c', e' \rangle :: e, s) \\
 K(\text{push}(c'); c, e, k) &= K(c, e, \langle c', e \rangle :: k) \\
 K(\text{store}(c'); c, e, k) &= K(c, \langle c', e \rangle :: e, k)
 \end{aligned}$$